

وثيقة المرجع المعماري (Architectural Reference Document — ARD)

نظام PropertyApi — منصة العقارات (Yaqeen Real Estate)

فهرس المحتويات

فهرس المحتويات.....	1
1. المقدمة.....	3
1.1 الغرض من الوثيقة.....	3
1.2 نطاق الوثيقة.....	3
1.3 تعريفات ومصطلحات واختصارات.....	4
1.4 المراجع.....	5
2. محركات المعمارية (Architectural Drivers).....	5
2.1 متطلبات العمل.....	5
2.2 متطلبات الجودة (Quality Attributes).....	6
2.3 القيود (Constraints).....	7
3. نظرة عامة على النظام (System Overview).....	8
3.1 وصف النظام.....	8
3.2 سياق النظام (System Context).....	8
3.3 الأهداف المعمارية.....	9
4. وجهات النظر المعمارية (Architectural Views).....	10
4.1 وجهة النظر المنطقية (Logical View).....	10
4.2 وجهة النظر التطويرية (Development View).....	11
4.3 وجهة النظر العملية (Process View).....	12
4.4 وجهة النظر الفيزيائية/النشر (Deployment View).....	12
4.5 وجهة النظر الأمنية (Security View).....	14
5. قرارات التصميم المعماري (Architectural Design Decisions).....	15

5.1 أنماط المعمارية المستخدمة.....	15
5.2 مبادئ التصميم.....	16
5.3 قرارات التكنولوجيا (مع التبرير).....	16
5.4 قرارات التكامل.....	16
5.5 سجلّ قرارات التصميم المعماري (Architecture Decision Records — ADR).....	17
ADR-001 — اعتماد Clean Architecture بأربع مشاريع فيزيائية مع اختبارات معمارية آلية.....	17
ADR-002 — رمز وصول قصير + رمز تحديث دّوار في كوكي HttpOnly مع كشف إعادة الاستخدام.....	17
ADR-003 — PostgreSQL مع امتداد PostGIS كقاعدة بيانات تشغيلية وحيدة.....	18
ADR-004 — Cloudinary كمزود وحيد لتخزين ومعالجة الوسائط.....	18
ADR-005 — عزل أحمال العمل على Redis (مُعاد صياغته من docs/architecture/adr-redis-workload-isolation.md).....	18
ADR-006 — المكتب العقاري (Agency) تجميع تنظيمي وليس عزل مستأجرين (Multi-Tenancy).....	19
ADR-007 — توكن CSRF مُعاد في جسم JSON، لا الاعتماد على قراءة كوكي عبر النطاقات فقط.....	19
ADR-008 — Render.com كمنصة استضافة، بيئتي Staging و Production معزولتين منطقيًا داخل مشروع واحد.....	20
6. اعتبارات الجودة (Quality Attribute Considerations).....	20
6.1 الأداء.....	20
6.2 قابلية التوسّع.....	21
6.3 الأمان.....	21
6.4 قابلية الصيانة.....	22
6.5 قابلية النشر والتعافي من الكوارث.....	22
7. المخاطر المعمارية (Architectural Risks).....	22
8. الخاتمة.....	23
8.1 الملخص.....	23
8.2 المراجعات المستقبلية.....	24
9. الملحق — المخططات التسلسلية للعمليات الجوهرية.....	24
9.1 المصادقة (Authentication).....	24
9.2 إدارة العقارات (Property Management).....	25
9.3 رفع الصور (Image Upload).....	26
9.4 التقييمات (Reviews).....	27
9.5 الزيارات (Visits).....	28

PropertyApi (.NET 8 Web API)	اسم النظام
1.0	إصدار الوثيقة
04-09-2026	تاريخ الإصدار
claude/agency- معتمدة — مطابقة للكود الفعلي في فرع ownership-logo-api	الحالة
Naeem-Ba/PropertyApi	المستودع
تحليل مباشر لشيفرة المصدر: 4 مشاريع رئيسية (API، Application، Domain، Infrastructure)، 30 وحدة تحكم، +37 ترحيلة قاعدة بيانات، ملفات النشر والمراقبة الفعلية	نطاق الفحص

1. المقدمة

1.1 الغرض من الوثيقة

الغرض من هذه الوثيقة هو توثيق المعمارية الفعلية لنظام **PropertyApi** كما هي مطبقة اليوم في الشيفرة المصدرية — وليس كما يُفترض أو يُخطّط لها. كل ادعاء تقني في هذه الوثيقة تم التحقق منه بالرجوع مباشرة إلى ملفات الكود (Program.cs، DependencyInjection.cs، وحدات التحكم، معالجات الأوامر/الاستعلامات، ملفات الإعداد، ملفات docker-compose)، وليس من وثائق تصميم منفصلة قد تكون قديمة.

الجمهور المستهدف:

الفئة	الفائدة من الوثيقة
مطوّرو الواجهة الخلفية (Backend) الجدد	فهم الطبقات، تدفقات العمل، ونقاط التمديد قبل الكتابة في الكود
مطوّرو الواجهة الأمامية (Angular SPA / تطبيق الجوال)	فهم عقود الـ API، حدود المصادقة، وقيود CORS/CSRF عبر البيئات
مهندسو DevOps / SRE	فهم بنية النشر، نقاط الفشل، واستراتيجيات المراقبة والتعافي
قادة تقنيون وأصحاب مصلحة في المنتج	فهم القيود المعمارية الحالية وتأثيرها على خارطة الطريق
مراجعو الأمان	فهم آليات المصادقة والتفويض وحدودها المعروفة

1.2 نطاق الوثيقة

ما تغطيه هذه الوثيقة:

- الواجهة الخلفية (.NET 8) PropertyApi بطبقاتها الأربع: (API) PropertyApi، PropertyApi.Application، PropertyApi.Domain، PropertyApi.Infrastructure.
- تكامل هذه الواجهة مع الأنظمة الخارجية: PostgreSQL/PostGIS، Redis، Cloudinary، Resend/SMTP، بوابة SMS، Google/Apple OAuth، Have I Been Pwned، مكدّس المراقبة (OpenTelemetry/Prometheus/Grafana/Tempo).
- بيئتا النشر الفعليّتان على Render (Production و Staging) و GitHub Actions كخط أنابيب CI/CD.

- خمس تدفقات عمل جوهرية موثقة بمخططات تسلسلية: المصادقة، إدارة العقارات، رفع الصور، التقييمات، الزيارات.

ما لا تغطيه هذه الوثيقة:

- الشيفرة الداخلية للواجهة الأمامية Angular SPA (مستودع Git منفصل تمامًا: -Naeem Ba/Wohnungsmieten) — تُذكر هنا فقط من زاوية العقد الذي تتبادله مع الـ (REST/JSON) API، الكوكيز، (CORS، CSRF).
- تطبيق الجوال (Capacitor) بتفاصيله الداخلية — يُعامل كعميل HTTP آخر للـ API.
- كل وحدات العمل الفرعية بتفصيل متساوٍ: التركيز الأعمق على المصادقة والعقارات والوسائط والتقييمات والزيارات (الوحدات المطلوبة صراحة)، مع نظرة عامة أخفّ على: المكاتب العقارية (Agencies)، الإقامات القصيرة (ShortStay)، سوق الخدمات (Services Marketplace)، والإدارة (Admin).

1.3 تعريفات ومصطلحات واختصارات

المصطلح	التعريف
ARD	Architectural Reference Document — وثيقة المرجع المعماري، هذه الوثيقة.
CQRS	Command Query Responsibility Segregation — فصل أوامر الكتابة عن استعلامات القراءة، مطبق هنا عبر MediatR.
DDD خفيف	Domain-Driven Design خفيف: كيانات ذات سلوك (Factory methods)، تحقق من قواعد العمل داخل الكيان) بدل كيانات بيانات فقيرة (Anemic).
JWT	JSON Web Token — رمز الوصول قصير الأمد (30 دقيقة).
Refresh Token	رمز تحديث طويل الأمد (30 يومًا)، مخزن في كوكي HttpOnly، مع كشف إعادة الاستخدام (Reuse Detection).
CSRF	Cross-Site Request Forgery — يُمنع هنا عبر توكن مزدوج (Double-Submit) مبني على Antiforgery المدمج في ASP.NET Core.
MediatR	مكتبة .NET تُطبّق نمط الوسيط (Mediator) لتوجيه كل أمر/استعلام إلى معالجه عبر أنبوب سلوكيات (Pipeline Behaviors).
EF Core	Entity Framework Core — ORM المستخدم للوصول إلى PostgreSQL عبر Npgsql.
PostGIS	امتداد PostgreSQL للبيانات الجغرافية المكانية (يُستخدم للبحث الجغرافي عن العقارات).
OTLP	OpenTelemetry Protocol — بروتوكول تصدير القياسات والتتبعات (Traces/Metrics) إلى مجمّع المراقبة.
SLO / SLI	Service Level Objective / Indicator — أهداف ومؤشرات مستوى الخدمة (انظر القسم 6.1).
RPO / RTO	Recovery Point/Time Objective — أقصى فقدان بيانات مسموح به وأقصى زمن تعافٍ (انظر القسم 6.5).
Wohnungsmieten	اسم مستودع الواجهة الأمامية Angular (منفصل عن هذا المستودع).

1.4 المراجع

الموقع	المراجع
docs/architecture/ARD/diagrams/*.puml	مخططات PlantUML المصاحبة لهذه الوثيقة
docs/architecture/adr-redis-workload-isolation.md	ADR: عزل أحمال Redis
docs/architecture/authentication-orchestration.md	معمارية تنسيق المصادقة (Auth Orchestration)
docs/architecture/email-confirmation-and-delivery.md	تأكيد البريد الإلكتروني والتسليم
docs/observability/slo-definition.md	تعريف أهداف مستوى الخدمة (SLO)
docs/operations/redis-ha-architecture.md	معمارية Redis عالي التوافر
docs/operations/rpo-rto.md	أهداف نقطة/زمن التعافي (RPO/RTO)
docs/password-policy.md	سياسة كلمة المرور
docs/performance/performance-test-runbook.md	دليل اختبار الأداء
docs/testing/staging-production-gate-checklist.md	قائمة تحقق بوابة الإصدار للإنتاج
ملحظة داخلية للفريق — أسماء الموارد والمعرفات في القسم 4.4	ملف تعريف بيئة Render الفعلية (Staging/Production)

2. محركات المعمارية (Architectural Drivers)

2.1 متطلبات العمل

PropertyApi هي الواجهة الخلفية الوحيدة لمنصة عقارية سورية/إقليمية (Yaqeen Real Estate) تخدم عدة نماذج عمل ضمن نظام واحد، مستخلصة من وحدات التحكم الثلاثين الفعلية:

وحدات التحكم ذات الصلة	الوظائف الأساسية	المجال الوظيفي
,AuthController ,PhoneAuthController ,PhonePasswordAuthController ,CsrfController	تسجيل/دخول بالبريد أو الهاتف، تسجيل دخول اجتماعي (Google/App)، استعادة كلمة المرور، OTP عبر SMS	الحسابات والمصادقة
,PropertiesController ,PropertyImagesController ,LocationSuggestionsController ,LookupsController	إنشاء/تعديل/نشر عقارات، تسعير، صور، بحث جغرافي، اقتراحات مواقع	إدارة العقارات (Listings)
,AgenciesController	تسجيل مكتب، عضوية، دعوات، نقل ملكية، شعار المكتب	المكاتب العقارية (Agencies)
,VisitsController ,ReviewsController ,MessagesController ,FavoritesController ,SavedSearchController	زيارات ميدانية، تقييمات، رسائل، مفضلات، بحث محفوظ	التفاعل بين الأطراف

المجال الوظيفي	الوظائف الأساسية	وحدات التحكم ذات الصلة
سوق الإقامة القصيرة (ShortStay)	وحدات إقامة، حجوزات، قواعد تسعير وحد أدنى للإقامة	,ShortStayListingsController ,ShortStayBookingsController ShortStayReviewsController
سوق مزود الخدمات	عروض خدمات (نقل، تشطيب، ...)، طلبات خدمة، تقييمات مزودين	,ServiceProvidersController ,ServiceOfferingsController ServiceRequestsController
الإدارة والتشغيل	لوحة تحكم إدارية، اشتراكات، صحة النظام، أدوات مراقبة اصطناعية	,AdminController ,OperationalController ObservabilitySyntheticController
البنية التحتية للمنتج	خطط الاشتراك، تحليلات، عملات وقوائم مرجعية	,PlansController ,AnalyticsController EnumController

متطلبات غير وظيفية بارزة مستخلصة من الكود نفسه (لا من نية مفترضة):

- **عدم كشف وجود الحساب (No Enumeration Oracle):** رسالة/رمز حالة تسجيل الدخول موحّد بغض النظر عن سبب الفشل (AuthController.Login، انظر السطر 140-157) — قرار أمني متعمّد موثّق بتعليق داخل الكود يشرح لماذا أزيل مسار "تحقق القفل" السابق.
- **صلاحية المستخدم مُشتقة من التوكن دائمًا، لا من جسم الطلب:** مكرّر في كل وحدة تحكم تقريبًا (OwnerId = GetCurrentUser(),)، وليس من DTO الوارد) — نمط أمني موحّد عبر كامل الكود.
- **قابلية إعادة المحاولة والتعافي من فشل جزئي:** رفع الصور يحذف الصور المرفوعة جزئيًا عند فشل صورة لاحقة ضمن نفس الطلب (Cleanup صريح في UploadPropertyImagesCommandHandler).

2.2 متطلبات الجودة (Quality Attributes)

الصفة	كيف تتحقق فعليًا في الكود	الدليل
الأمان (Security)	JWT + Refresh Token مدوّر مع كشف إعادة الاستخدام، CSRF بتوكن مزدوج، رؤوس أمان HTTP، HSTS، تحديد معدل مبني على Redis لكل نقطة نهاية حساسة، فحص كلمات المرور مقابل Have I Been Pwned، تحقق من توقيع الصور الثنائي (magic bytes) وليس فقط امتداد الملف	,Program.cs (ترتيب الوسيطات)، ,PasswordSecurityService.cs UploadPropertyImagesCommandHandler.cs
الأداء (Performance)	هدف 95% SLO: من الطلبات $\geq 500ms$ على نافذة 30 يومًا؛ تخزين مؤقت للمخرجات (Output Cache) عبر Redis؛ تخزين مؤقت لتسريع فحص Security Stamp	,docs/observability/slo-definition.md OutputCacheRegistration.cs
قابلية التوسّع (Scalability)	بدون حالة على مستوى العملية (Stateless): الجلسات في JWT/Redis لا في ذاكرة العملية؛ SignalR backplane عبر Redis يسمح بأكثر من نسخة خلفية؛ Output Cache وتحديد المعدل موزّعان عبر Redis	,Program.cs (AddPropertyApiSignalR) env.example (SignalR__Provider).

الصفة	كيف تتحقق فعليًا في الكود	الدليل
الموثوقية والتعافي (Reliability/DR)	RPO ≤ 60 دقيقة، RTO ≤ 120 دقيقة (مُقاسة فعليًا بسير عمل تعافي آلي أسبوعي)؛ Redis HA عبر Sentinel (Primary/Replica + 3 Sentinels) مُختبَر في CI	docs/operations/rpo-rto.md, ci/docker-compose.production-gate.yml
قابلية الصيانة (Maintainability)	فصل صارم بين الطبقات (Clean Architecture)؛ اختبارات معمارية مخصصة (PropertyApi.Architecture.Tests) تفرض القيود آليًا في CI؛ نمط CQRS موحد لكل وحدة عمل جديدة	tests/PropertyApi.Architecture.Tests
قابلية النشر (Deployability)	صورة حاوية واحدة (multi-stage Dockerfile) على أساس aspnet:8.0- (سطح هجوم مخفّض)؛ بوابة إصدار آلية (production-gate.yml) قبل أي نشر إلى Staging	PropertyApi/Dockerfile, ci/docker-compose.production-gate.yml
إمكانية المراقبة (Observability)	OpenTelemetry مدمج منذ أول سطر في Program.cs (AddPropertyApiObservability)؛ تصدير OTLP/gRPC إلى مجمّع يَغْذِي Prometheus/Grafana/Tempo	Program.cs:19, observability/docker-compose.observability.yml

2.3 القيود (Constraints)

القيد	الأثر المعماري
C. / NET 8 # فقط، بنية Clean Architecture من أربع مشاريع	كل تبعية جديدة يجب أن تحترم اتجاه الاعتماد (API→Application→Domain) Infrastructure→Application/Domain؛ يُفرض آليًا عبر PropertyApi.Architecture.Tests
PostgreSQL + PostGIS كقاعدة بيانات وحيدة	كل ميزة بحث جغرافي (قرب الموقع، ضمن نطاق) يجب أن تُبنى على وظائف PostGIS، لا على محرك بحث منفصل.
Cloudinary كمخزن وسائط وحيد	لا يوجد تجريّد "متعدد المزودين" فعلي؛ StagingSmokeMediaStorageService هو بديل اختباري فقط لا مزود إنتاجي بديل.
مستودعان منفصلان تمامًا للواجهتين الخلفية والأمامية	أي تغيير في عقد HTTP (شكل JSON، اسم كوكبي، رأس CSRF) يتطلب تنسيقًا يدويًا بين مستودعي Git مختلفين — لا يمكن لأي أداة فحص نوع مشترك أن تكتشف عدم التطابق.
Netlify (أمامية) + Render (خلفية) لا يشتركان بنفس النطاق (Domain) في staging/production	يمنع الاعتماد على قراءة كوكبي عبر النطاقات من جافاسكربت؛ يفرض إرجاع توكن CSRF في جسم JSON بدل الاعتماد على الكوكبي وحدها (انظر القسم 4.5).
خطة "Free Render" لبيئة Staging	بدون Shell access، بدون نسخ احتياطي لقاعدة بيانات Staging، صلاحية قاعدة البيانات ~30 يومًا فقط؛ ترحيلات EF Core لا تُطبّق تلقائيًا عند الإقلاع في أي بيئة — يجب تشغيلها يدويًا عبر tools/PropertyApi.Migrator
Cloudinary بخطة مجانية (بيئة منتج واحدة)	عزل Staging/Production لبيانات الوسائط هو عزل بيانات اعتماد (API keys) فقط، وليس عزل تخزين فعلي — يرفع كلا

القيد	الأثر المعماري
	البيئتين إلى نفس المجلدات المُبرمجة صراحة في الكود (... /property-images).
حساب Resend بنطاق موثّق واحد (realestateworld.world)	فصل Staging عن Production بالبريد الإلكتروني يتم عبر مفتاح API مختلف على نفس النطاق، لا عبر نطاق منفصل.

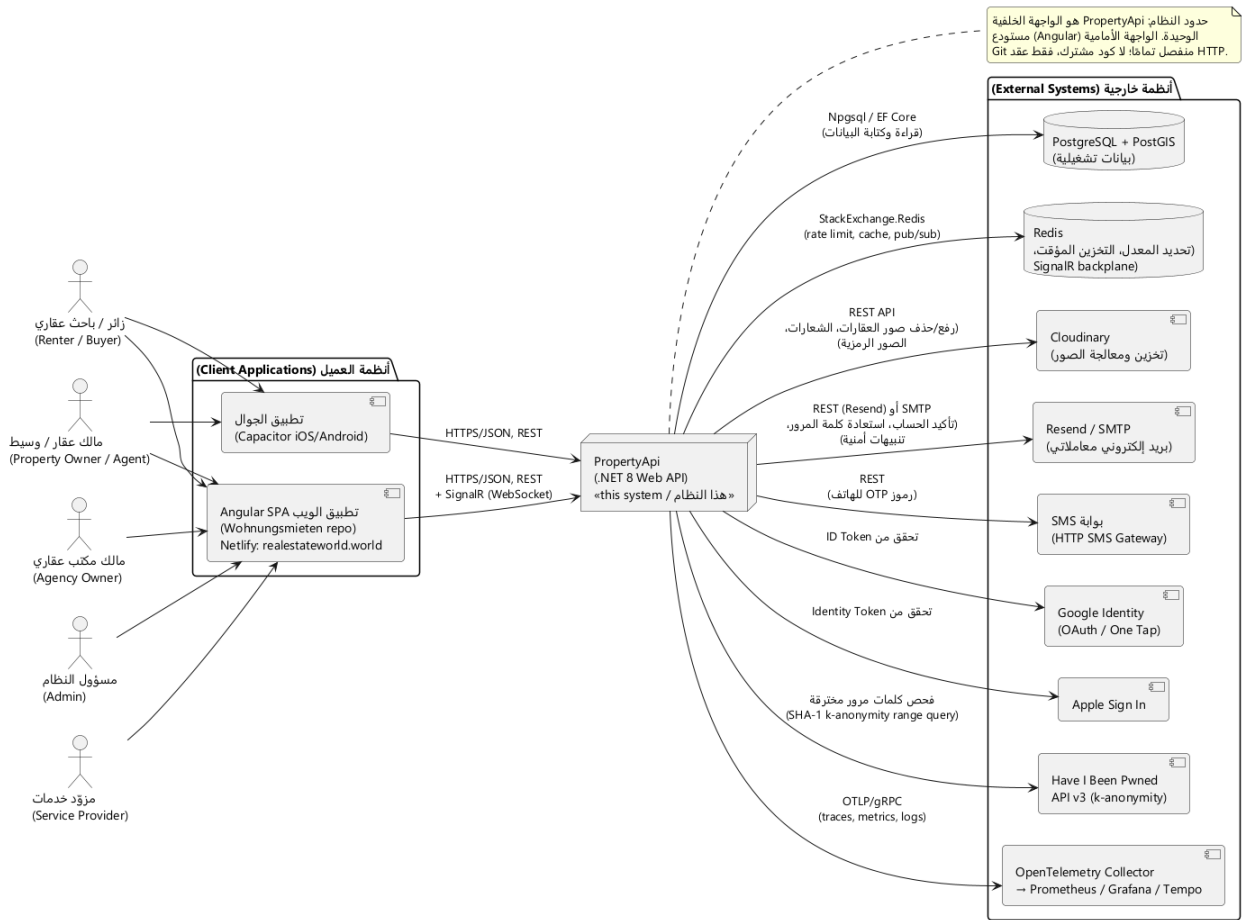
3. نظرة عامة على النظام (System Overview)

3.1 وصف النظام

PropertyApi واجهة خلفية REST مبنية بـ **NET 8** تخدم منصة عقارية متعددة الأدوات: باحثون عن عقارات (إيجار/شراء)، مَلّاك ووسطاء أفراد، مكاتب عقارية مسجّلة (Agencies)، مزودو خدمات مرتبطة بالعقار (نقل، تشطيب، صيانة)، ومسؤولو نظام. تُبنى المعمارية على نمط **Clean Architecture** بأربع طبقات فيزيائية منفصلة (مشاريع NET. مستقلة) مع **CQRS** عبر **MediatR** داخل طبقة التطبيق، وتُخزّن البيانات التشغيلية في **PostgreSQL** بامتداد **PostGIS** للاستعلامات الجغرافية.

النظام بلا حالة على مستوى العملية (stateless compute): أي حالة يجب أن تنجو بين الطلبات (الجلسة، معدّل الطلبات، ذاكرة التخزين المؤقت للمخرجات) تُحفظ في **PostgreSQL** أو **Redis**، لا في ذاكرة عملية NET. — وهو ما يسمح بتشغيل أكثر من نسخة من الـ API خلف موازن تحميل دون فقدان الاتساق (مثال: SignalR backplane عبر Redis عند الحاجة لأكثر من نسخة).

3.2 سياق النظام (System Context)



*(المصدر القابل للتعديل: diagrams/01-system-context.puml)

يوضح المخطط أعلاه أن **PropertyApi هو النظام الوحيد** الذي يملك منطق العمل وقاعدة البيانات التشغيلية. الواجهة الأمامية (Angular SPA) وتطبيق الجوال (Capacitor) عميلان لا يحملان أي منطق عمل مستقل — كل قرار (من يملك ماذا، من يجوز له فعل ماذا) يتخذ في الخلفية. الأنظمة الخارجية الثمانية المرسومة كلها **تبعيات صادرة** (النظام يستدعيها)، ولا يوجد نظام خارجي يستدعي PropertyApi (لا Webhooks واردة موثقة في هذا الإصدار).

3.3 الأهداف المعمارية

استخلاصًا من محركات المعمارية في القسم 2، تسعى المعمارية إلى:

1. **حماية سلامة البيانات المالية والقانونية** (أسعار العقارات، تراخيص المكاتب العقارية) عبر تحقق صارم على مستوى النطاق (Domain) لا يمكن تجاوزه من طبقة API.
2. **منع تسريب معلومات عبر قنوات جانبية** (استجابات تسجيل الدخول، رسائل الأخطاء) — مبدأ مطبق بانضباط ملحوظ في الكود الفعلي، وليس شعارًا نظريًا.
3. **إبقاء التوسع الأفقي ممكنًا دون إعادة تصميم** عبر عدم حفظ أي حالة في ذاكرة العملية.
4. **جعل كل قرار تفويض قابلاً لإعادة الاستخدام ومركزيًا** (مثال: `IPROPERTYOwnershipService.EnsureOwnerAsync / GetOwnedPropertyOrThrowAsync` تُستدعى من كل معالج يلمس عقارًا، بدل تكرار منطق "هل أنت المالك؟" في كل مكان).
5. **إبقاء الملاحظة (Observability) جزءًا من الإقلاع لا إضافة لاحقة** — `AddPropertyApiObservability()` هو أول سطر تنفيذي في `Program.cs`.

4. وجهات النظر المعمارية (Architectural Views)

4.1 وجهة النظر المنطقية (Logical View)

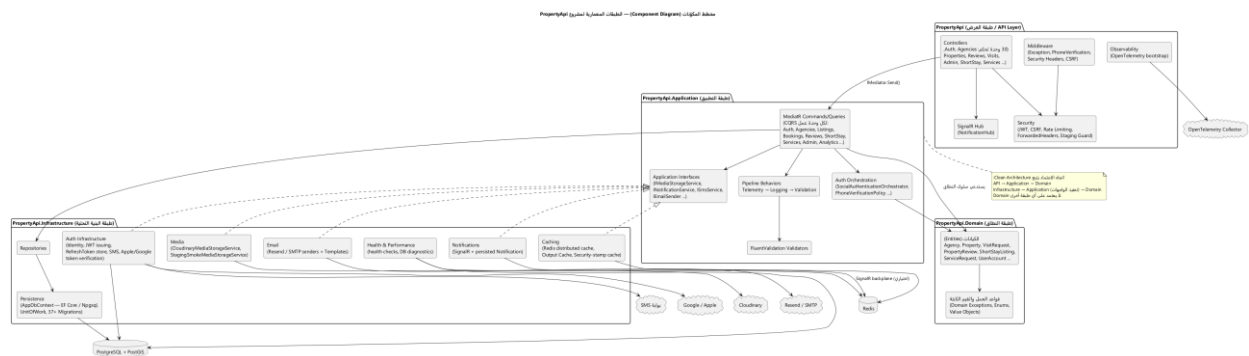
النظام مقسّم إلى أربع طبقات فيزيائية (مشاريع .NET منفصلة، لا مجرد مجلدات) تفرض اتجاه الاعتماد التالي:

PropertyApi (API) —depends on—▶ PropertyApi.Application —depends on—▶ PropertyApi.Domain

PropertyApi.Infrastructure —implements interfaces of—▶ PropertyApi.Application

PropertyApi.Infrastructure —depends on—▶ PropertyApi.Domain

PropertyApi.Domain لا يعتمد على أي مشروع آخر — لا حزم ASP.NET، لا EF Core، لا MediatR. هذا القيد مفروض آليًا عبر tests/PropertyApi.Architecture.Tests، وليس اتفاقًا غير مكتوب.



(المصدر القابل للتعديل: diagrams/03-component.puml)

تفصيل كل طبقة:

الطبقة	المسؤولية	أمثلة فعلية من الكود
PropertyApi (API)	استقبال HTTP، تحويل DTO ↔ أمر/استعلام MediatR، الوسيطات (Middleware)، الأمان الحدودي (JWT، CSRF)، رؤوس الأمان، تحديد المعدّل	30 وحدة تحكم: Security/Middleware/Program.cs
PropertyApi.Application	منطق التنسيق (Orchestration) بلا معرفة بـ ASP.NET أو EF Core أوامر/استعلامات MediatR؛ تحقق FluentValidation واجهات (Interfaces) للبنية التحتية	Auth/Orchestration/SocialAuthenticationOrchestrator.cs */Listings/Commands

الطبقة	المسؤولية	أمثلة فعلية من الكود
PropertyApi.Domain	الكيانات ذات السلوك، قواعد العمل الثابتة، الاستثناءات المتخصصة (DomainException)	Agencies/Entities/Agency.cs (منطق) Create/UpdateProfile/TransferOwnership داخل الكيان نفسه، لا في المعالج
PropertyApi.Infrastructure	تنفيذ الواجهات: EF، Core/Npgsql، Cloudinary، Resend/SMTP، Redis، SignalR، بوابة SMS، Google/Apple token verification	(DbSet + { }) Persistence/AppDbContext.cs Media/CloudinaryMediaStorageService.cs

نمط CQRS الموحد لكل وحدة عمل (مطبّق حرفيًا في كل وحدة من الوحدات الثلاثين تقريبًا):

```
Controller → IMediator.Send(Command|Query)
    → TelemetryBehavior → LoggingBehavior → ValidationBehavior (FluentValidation)
    → Handler (IRequestHandler<TRequest, TResponse>)
    → Domain Entity (تحقق قواعد العمل) + Repository/UnitOfWork
```

هذا الترتيب (Telemetry → Logging → Validation → Handler) مُسجّل حرفيًا في PropertyApi.Application/DependencyInjection.cs، وهو ثابت لكل أمر/استعلام في النظام دون استثناء.

4.2 وجهة النظر التطويرية (Development View)

تنظيم الشيفرة والحزم:

- حل واحد (PropertyApi.sln) يضم 4 مشاريع إنتاجية + 8 مشاريع اختبار + 3 أدوات مستقلة (/tools).
- إدارة حزم مركزية (Central Package Management) عبر Directory.Packages.props — إصدار واحد لكل حزمة عبر كامل الحل، يمنع تعارض الإصدارات بين المشاريع.
- أقفال استعادة محكمة (packages.lock.json لكل مشروع) وdotnet restore --locked-mode — يمنع أي تغيير غير مقصود في شجرة التبعية عند البناء.

مشاريع الاختبار الثمانية وأدوارها:

المشروع	الغرض
PropertyApi.Application.Tests	اختبارات وحدة لمعالجات الأوامر/الاستعلامات (Mocking للواجهات)
PropertyApi.Architecture.Tests	يفرض قيود المعمارية آليًا (اتجاه الاعتماد، Endpoints العامة الموافق عليها، إلخ) — بوابة CI حقيقية، ليست اختبارًا توثيقيًا
PropertyApi.Auth.Tests	تغطية مركزة على وحدة المصادقة عبر طبقاتها الأربع
PropertyApi.Concurrency.Tests	سيناريوهات تزامن (مثال: طلبين متزامنين على نفس المورد)
PropertyApi.Integration.Tests	اختبارات تكامل مع قاعدة بيانات Redis/حقيقيين (WebApplicationFactory)
PropertyApi.Observability.Tests	يتحقق من أن القياسات/التبعية تُصدّر فعليًا لأن الكود "يفترض" أنها تُصدّر

المشروع	الغرض
PropertyApi.Performance.Tests	اختبارات أداء مقابل خط أساس محفوظ
PropertyApi.StagingSmokeTests	اختبارات دخان فعلية ضد بيئة Staging المنشورة (وليست Mock)

بناء الحاوية (PropertyApi/Dockerfile): بناء متعدد المراحل (sdk:8.0) للبناء → aspNet:8.0-jammy- chiseled-extra (للتشغيل). صورة التشغيل **chiseled** (مقتطعة): بلا shell، بلا مدير حزم، مستخدم غير جذري (APP_UID\$) — تقليل سطح الهجوم قرار معماري صريح، لا افتراضي.

4.3 وجهة النظر العملية (Process View)

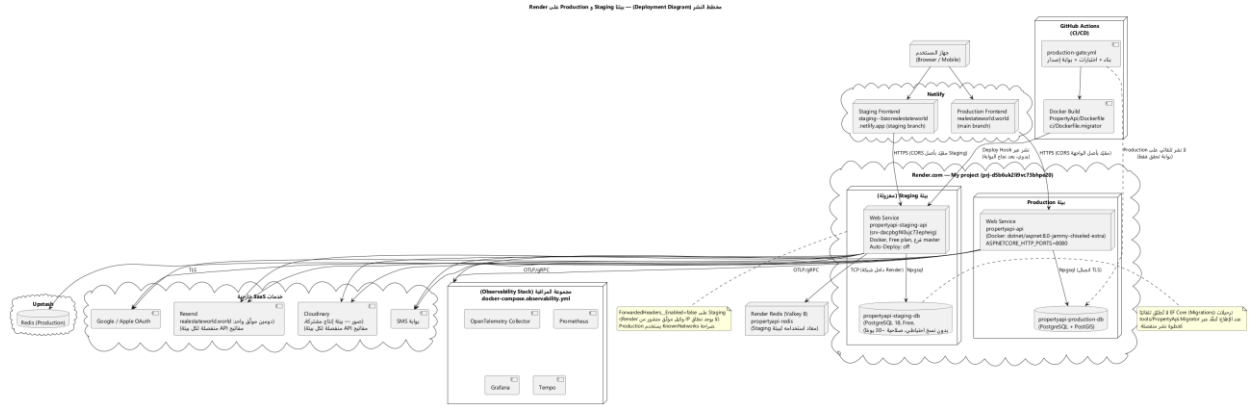
انظر المخططات التسلسلية الخمسة الكاملة في الملحق (القسم 9) — هنا ملخص لآلية معالجة الطلب العامة وترتيب الوسيطات (Middleware) الفعلي كما هو مسجل حرفياً في Program.cs:

1. UseForwardedHeaders (الحقيقية خلف الوكيل، إن كانت موثوقة IP ثقة بعناوين)
2. UsePerformanceInstanceHeader / UsePerformanceDatabaseDiagnostics
3. UsePropertyApiObservability (Trace Span بدء نطاق التتبع)
4. UseHsts (Production) | UseHttpsRedirection (غير ذلك)
5. ExceptionHandlingMiddleware (موحدة HTTP تحويل الاستثناءات إلى استجابات)
6. UsePropertyApiSecurityHeaders (CSP، X-Frame-Options، إلخ)
7. UseSwagger/UseSwaggerUI (Development/Testing/CI أو Staging فقط، بعلوم صريح)
8. UseStaticFiles
9. UseRouting
10. UseCors("DefaultCors")
11. UseAuthentication (إن وُجد JWT فك تشفير)
12. PhoneVerificationRestrictionMiddleware
13. UseRedisRateLimiting | UseRateLimiter (Redis حسب توفر)
14. UseCookieCsrfProtection (refresh_token غير الآمنة، عند وجود كوكي POST/PUT/DELETE لطلبات)
15. UseAuthorization ([Authorize]/[Authorize(Roles=...)] فحص)
16. UseOutputCache
17. MapOperationalHealthEndpoints / MapControllers / MapHub("/notificationHub")

ملاحظة معمارية مهمة: المصادقة (خطوة 11) تسبق تحديد المعدّل (خطوة 13) وتسبق CSRF (خطوة 14) — أي أن هوية المستخدم معروفة قبل تطبيق أي قيد إضافي، وهو ما يسمح لسياسات تحديد المعدّل مستقبلاً بالتمييز بحسب المستخدم لا بحسب IP فقط (غير مطبق حالياً لكل السياسات، لكن الترتيب يسمح به).

إدارة الأخطاء غير الحرجة (نمط مكرّر عبر عشرات المعالجات): أي عملية "ثانوية" (إرسال إشعار، اقتراح موقع جديد) تُلفّ صراحة بـ try/catch منفصل عن حفظ البيانات الأساسي، مع تسجيل الخطأ (logger.LogError) دون إسقاط الطلب بأكمله. هذا النمط مسمّى صراحة "Fail Fast" للتحقق، لكن Best-Effort للتبعيات الثانوية" في تعليقات الكود نفسها (مثال: UpdatePropertyCommandHandler.NotifyChangesAsync، RequestVisitCommandHandler.AddReviewCommandHandler).

4.4 وجهة النظر الفيزيائية/النشر (Deployment View)



*(المصدر القابل للتعديل: diagrams/02-deployment.puml)

البيئتان الفعليتان الحيّتان (Render.com، مشروع واحد prj-d5b6uk21i9vc73bhpe20، بيئتان منفصلتان):

Staging	Production	
propertyapi-staging-api (srv-(dacpbgf40ujc73epheig	propertyapi-api	خدمة الويب
propertyapi-staging-db (PostgreSQL 18، خطة Free، بلا نسخ احتياطي، صلاحية ~30 يومًا)	PostgreSQL + PostGIS مخصّصة	قاعدة البيانات
propertyapi-redis (Valkey 8) (Render، مُعاد استخدامه)	Upstash (مُدار خارجيًا)	Redis
staging--bizorealestateworld.netlify.app (فرع staging)	realstateworld.world (Netlify) (main فرع)	الواجهة الأمامية
مُعطّل (Auto-Deploy: off) — نشر عبر Deploy Hook يدويًا بعد نجاح production-gate.yml	— (بوابة تحقق فقط، لا نشر آلي مباشر موثّق في هذه الوثيقة)	النشر التلقائي
يدوية عبر tools/PropertyApi.Migrator (لا تطبيق تلقائي عند الإقلاع في أي بيئة)	يدوية عبر tools/PropertyApi.Migrator	ترجيلات قاعدة البيانات
معطّلة (ForwardedHeaders__Enabled=false) — Render لا ينشر نطاق IP وكيل موثّق، ولا تتوفر صلاحية Shell على الخطة المجانية للتحقق التجريبي	مفعّلة، مع KnownNetworks صريحة	رؤوس الوكيل الموثوقة (ForwardedHeaders)

أثر معماري مباشر لتعطيل ForwardedHeaders على Staging: تقسيم تحديد المعدّل حسب عنوان IP للعميل (RedisRateLimitPartitionKeyResolver) يرى عنوان IP الحافة الواحد لـ Render بدل عنوان كل زائر فعلي — أي أن حصص تحديد المعدّل (مثال: 5 محاولات تحقق OTP/15 دقيقة) تُشارك فعليًا بين كل زوّار Staging في آن واحد. هذا خطر تشغيلي معروف ومقبول بوعي (وليس عيبًا غير مكتشف) — مسجّل صراحة كقرار مؤرّخ من الفريق، ويُعاد النظر فيه فقط عند الحاجة لاختبار متزامن حقيقي أو ترقية خطة Render.

خط أنابيب (CI/CD) (GitHub Actions)

سير عمل production-gate.yml يبنّي صورة الحاوية، يشغّل الاختبارات (بما فيها اختبارات المعمارية والتكامل)، ويشغّل طبولوجيا **Redis Sentinel HA كاملة** (Primary + Replica + 3 Sentinels) على شبكة Docker معزولة

بعناوين IP ثابتة) في مهمة CI مخصصة — لا تُشغل هذه الطبولوجيا في كل بناء عادي، بل فقط عند استدعائها صراحة، حفاظًا على زمن بناء كل PR عادي.

4.5 وجهة النظر الأمنية (Security View)

المصادقة (Authentication):

- **JWT** قصير الأمد (30 دقيقة افتراضيًا، قابل للتهيئة عبر `Jwt:AccessTokenMinutes` + `Refresh Token` طويل الأمد (30 يومًا) مخزن في **كوكي HttpOnly** — لا يصل جسم استجابة JSON لتسجيل الدخول لأي رمز تحديث بعد الإصلاح الموثق في الكود (RELEASE-BLOCKERS-AR.md B-13).
- **كشف إعادة استخدام رمز التحديث (Refresh Token Reuse Detection):** عند اكتشاف استخدام رمز مُبطل سابقًا، يُبطل النظام كل الرموز النشطة لذلك المستخدم، ويُدرّ **SecurityStamp** (ما يُبطل أيضًا كل Access Token صادر مسبقًا لذلك المستخدم فورًا)، ويُسجل حدث تدقيق (RefreshTokenReuseDetected) — استجابة كاملة لسيناريو سرقة رمز، لا مجرد رفض الطلب الحالي.
- **تسجيل دخول اجتماعي:** Google (ID Token) و Apple (Identity Token + nonce) خام يُتَحَقَّق منه صراحة — إصلاح موثق (B-16)، عبر `SocialAuthenticationOrchestrator` الذي يفصل التحقق من الهوية (`SocialIdentityValidator`) عن حل الحساب (`SocialAccountResolver`) عن تنفيذ التغيير الفعلي (`SocialAccountMutationCoordinator`).
- **مصادقة عبر الهاتف (Phone OTP):** رموز تُرسل عبر بوابة SMS قابلة للتهيئة، بسياسة تحديد معدّل صارمة (3 إرسالات/15 دقيقة، 5 محاولات تحقق/15 دقيقة).

التفويض (Authorization):

- أدوار `ASP.NET Identity: User, Agent, Admin, AgencyOwner, AgencyAgent` **الدور ليس كافيًا وحده** **أبداً لمكاتب عقارية أو عقارات:** كل معالج يقارن أيضًا معرّف المستخدم المستدعي بـ `OwnerId/OwnerUserId` الفعلي على الكيان المستهدف (موثق صراحة في تعليق رأس `"AgenciesController: "holding the role says a user owns *an* agency, not *this* one"`).
- عزل الملكية على العقارات مركزي عبر خدمة واحدة (`IPROPERTYOwnershipService`)، لا منطق مكرّر لكل نقطة نهاية.
- **حماية CSRF:** توكن مزدوج التقديم (Double-Submit) مبني على `IAntiforgery` المدمج، مع تعديل جوهري لأن الواجهة الأمامية والخلفية لا يشتركان بنفس النطاق المسجل في staging/production (`realestateworld.world` أمام `onrender.com`): يُعاد توكن الطلب في جسم استجابة **JSON** (`api/security/csrf-token/`) بالإضافة إلى كوكي قابل للقراءة من جافاسكربت، لأن الاعتماد فقط على قراءة كوكي عبر نطاقات غير مشتركة لا يعمل أصلًا من متصفح — قرار موثق ومُختبر عبر مستودعي الواجهتين (انظر ADR-007 في القسم 5).

تحديد المعدّل (Rate Limiting): مبني على Redis في الإنتاج (نافذة ثابتة/Fixed Window)، بسياسة **مستقلة لكل نقطة نهاية حساسة** — جدول كامل في القسم 6.3.

رؤوس أمان HTTP و HSTS: **HSTS:** بمدة سنة كاملة (`MaxAge = 365` يومًا)، `IncludeSubDomains, Preload` — **مفعّل بصرامة كاملة** في الإنتاج فقط (لا يُطبّق `UseHsts` خارج `Production`: البيئات الأخرى تستخدم `UseHttpsRedirection` العادي).

حماية كلمة المرور: فحص محلي لقواعد التعقيد + فحص k-anonymity مقابل **Have I Been Pwned API v3** (بإدئة SHA-1 فقط تُرسل، لا كلمة المرور نفسها) عبر PasswordSecurityService، بقاطع دائرة (PwnedPasswordsCircuitBreaker) يوقف الاستدعاءات الخارجية بعد سلسلة فشل متتالية بدل تعليق كل تسجيل حساب جديد إذا كانت خدمة HIBP نفسها معطلة.

حماية الملفات المرفوعة: لا يُكتفى بامتداد الملف أو Content-Type المُعلن؛ يُتحقق من **التوقيع الثنائي الحقيقي (Magic Bytes)** لأول 8-12 بايت من الملف قبل قبوله (JPEG/PNG/WebP) — يمنع رفع ملف تنفيذي بامتداد png. مزيف.

القيد الأمني المعروف والمقبول صراحة (وليس عيبًا مخفيًا): المكاتب العقارية (Agencies) **تجميع تنظيمي، لا عزل مستأجرين (Multi-Tenancy) حقيقي** — لا يوجد Global Query Filter على AgencyId، وبيانات أكثر من مكتب تعيش في نفس الجداول مفصولة بمُسندات (Predicates) صريحة فقط. هذا موثّق حرفيًا في تعليق رأس ملف Agency.cs كقرار نطاق واعٍ، مع مسار ترقية معروف (Global Query Filter + IAgencyContext) سياقي إن استُدعيت الحاجة لعزل صارم مستقبلاً.

5. قرارات التصميم المعماري (Architectural Design Decisions)

5.1 أنماط المعمارية المستخدمة

النمط	أين يُطبّق	لماذا
Clean Architecture (طبقات متّجهة الاعتماد)	كامل الحل، 4 مشاريع فيزيائية	يعزل قواعد العمل عن أي تفصيل تقني (قاعدة بيانات، إطار ويب) قابل للاستبدال
CQRS عبر MediatR	كل وحدة عمل في PropertyApi.Application	يفصل مسار الكتابة (تحقق + تغيير حالة) عن مسار القراءة (تحسين أداء الاستعلام دون قيود الكتابة)؛ نقطة تمديد موحّدة (Pipeline Behaviors)
Domain Model غني (Rich Domain Model) ، لا Anemic Domain Model	الكيانات في PropertyApi.Domain	قواعد العمل (مثال: تحقق طول الاسم، صيغة ال Slug، حالة الدورة الزمنية للزيارة) تعيش داخل الكيان (Agency.Create, VisitRequest) لا مبعثرة في (state machine) المعالجات
Repository + Unit of Work	PropertyApi.Infrastructure/Repositories/Persistence/UnitOfWork.cs	يجمّع كل تغييرات الطلب الواحد في معاملة قاعدة بيانات واحدة (SaveChangesAsync) واحد لكل طلب (في الغالب)
Best-Effort للتبعيات الثانوية (إشعارات، اقتراحات)	متكرر عبر عشرات المعالجات	يمنع فشل SignalR أو بريد إلكتروني من إسقاط عملية أساسية محفوظة أصلاً بنجاح
Event-Driven جزئي عبر SignalR	NotificationHub, INotificationService	إشعارات فورية (زيارة جديدة، تقييم جديد) دون Polling من الواجهة الأمامية

5.2 مبادئ التصميم

6. لا تثق بأي معرّف حسّاس قادم من العميل — OwnerId/RequestingUserId/ReviewerId تُشتق دائمًا من مُطالبة (Claim) داخل JWT الموقّع، لا من جسم الطلب.
7. مركزية منطق التفويض بدل تكراره — IPropertyOwnershipService مثال متكرر الاستشهاد به داخل الكود نفسه كمرجع للنمط الصحيح.
8. الفشل السريع للتحقق، التسامح للتبعيات الثانوية (Fail Fast vs. Best Effort) — مبدأ موثّق صراحة بالاسم في تعليقات عدة معالجات.
9. ترتيب الفحوصات بحسب التكلفة، لا بحسب الأهمية فقط — مثال حرفي موثّق في AddReviewCommandHandler: التحقق من وجود العقار (استعلام رخيص) يسبق التحقق من زيارة مكتملة (Join أكبر).
10. عدم تسريب معلومات عبر فرق في الاستجابة — تسجيل الدخول الفاشل، تحديدًا، لا يكشف سبب الفشل.

5.3 قرارات التكنولوجيا (مع التبرير)

التقنية	الدور	التبرير المستخلص من الاستخدام الفعلي
NET 8 / ASP.NET Core.	منصة التنفيذ	دعم طويل الأمد (LTS)، أداء عالٍ، نظام Middleware ناضج لبناء أمان حدودي مخصّص
PostgreSQL + PostGIS	قاعدة البيانات التشغيلية الوحيدة	حاجة فعلية لاستعلامات جغرافية (قرب/نطاق) على العقارات، لا يمكن لقاعدة بيانات علائقية عادية أو NoSQL بسيطة تقديمها بنفس الكفاءة دون محرك بحث منفصل
EF Core / Npgsql	ORM	نضج، دعم Migrations، تكامل جيد مع PostGIS عبر NetTopologySuite
MediatR + FluentValidation	نمط CQRS والتحقق	فصل التحقق عن منطق العمل؛ نقطة تمديد موحّدة عبر Pipeline Behaviors
Redis (StackExchange.Redis)	تحديد المعدّل، التخزين المؤقت، SignalR، backplane، تسريع Security Stamp	حاجة فعلية لحالة مشتركة عبر أكثر من نسخة عملية (Stateless compute)
Cloudinary	تخزين ومعالجة الصور (عقارات، صور رمزية، شعارات مكاتب)	تحويل/تحسين صور جاهز دون بناء خط أنابيب معالجة صور داخلي
Resend (أساسي) / SMTP (بديل)	بريد إلكتروني معاملاتي	Resend مزوّد حديث بواجهة REST بسيطة؛ SMTP يبقى مسارًا احتياطيًا مُهيّأً صراحة
SignalR	إشعارات فورية	يستبدل الحاجة لاستطلاع دوري (Polling) من الواجهة الأمامية لحالة الزيارات/التقييمات/الرسائل
+ OpenTelemetry (OTLP) Prometheus + Grafana + Tempo	المراقبة الكاملة (Traces/Metrics/Logs)	معيار مفتوح، غير مقيد بمزوّد سحابي واحد، يسمح باستبدال الخلفية (Backend) لاحقًا دون تغيير كود القياس
Docker (صورة chiseled)	التغليف والنشر	تقليل سطح الهجوم (بلا shell، بلا مدير حزم في صورة التشغيل) قرار أمني صريح لا افتراضي

5.4 قرارات التكامل

- **تكامُل Cloudinary:** عبر `IMediaStorageService` (واجهة في `Application`) — `CloudinaryMediaStorageService` (تنفيذ في `Infrastructure`). كل استدعاء رفع يُرفَق بمجلد منطقي (`property-images`, إلخ)، ويُنظَف (`DeleteImageAsync`) عند فشل جزئي ضمن نفس الطلب.
- **تكامُل البريد الإلكتروني:** مزوّد قابل للتبديل عبر (`Console|Smtplib|Resend`) `Email:Provider` `Console`. مرفوض صراحة في الإنتاج (يمنع "نسيان" ضبط مزوّد حقيقي قبل النشر).
- **تكامُل SMS:** مزوّد HTTP عام قابل للتهيئة بالكامل (`SmsProvider:*`), مع تحقق إلزامي في الإنتاج من HTTPS ومفتاح API ورقم المرسل.
- **تكامُل المراقبة:** تصدير واحد فقط (`OTLP/gRPC`) إلى مجمّع وسيط، لا تكامل مباشر مع أي `SaaS` مراقبة تجاري — يبقي الباب مفتوحًا لاستبدال `Prometheus/Grafana/Tempo` بأي بديل يقبل `OTLP`.

5.5 سجلّ قرارات التصميم المعماري (ADR — Architecture Decision Records)

القرارات التالية مُستخلصة ومُصاغة رسميًا من قرارات فعلية موثّقة في الكود وتعليقاته وملفات `/docs`، وليست افتراضات. `ADR-005` يعيد صياغة قرار موجود مسبقًا بصيغة رسمية في `docs/architecture/adr-redis-workload-isolation.md`.

ADR-001 — اعتماد Clean Architecture بأربع مشاريع فيزيائية مع اختبارات معمارية آلية

- **الحالة:** مقبول ومطبّق بالكامل.
- **السياق:** نظام متعدد الوحدات (عقارات، مكاتب، إقامة قصيرة، خدمات، إدارة) ينمو باستمرار؛ خطر انزلاق منطق العمل إلى طبقة الويب أو تسرّب تفاصيل `EF Core` إلى طبقة النطاق يزداد مع الفريق والزمن.
- **القرار:** فصل فيزيائي (لا مجرد `Namespace`) إلى 4 مشاريع `.NET`، مع مشروع اختبار مخصّص (`PropertyApi.Architecture.Tests`) يفرض اتجاه الاعتماد وقواعد أخرى (مثل قائمة نقاط النهاية العامة الموافق عليها) كبوابة `CI` حقيقية تُفشل البناء عند الانتهاك.
- **البدائل المرفوضة:** توثيق نصّي لقواعد المعمارية بلا فرض آلي (رُفض: يتآكل مع الزمن دون إنفاذ)؛ معمارية طبقة واحدة (`Monolith` بلا فصل) (رُفض: يصعب اختبار منطق العمل بمعزل عن `ASP.NET/EF Core`).
- **الأثر:** كل ميزة جديدة أبطأ قليلًا في الكتابة الأولى (يجب المرور عبر 3-4 طبقات) مقابل قابلية اختبار وصيانة أعلى بكثير على المدى الطويل.

ADR-002 — رمز وصول قصير + رمز تحديث دوّار في كوكي `HttpOnly` مع كشف إعادة الاستخدام

- **الحالة:** مقبول ومطبّق.
- **السياق:** الحاجة لتقليل نافذة الخطر عند تسرّب رمز وصول (`Access Token`) مع تجنّب إجبار المستخدم على تسجيل الدخول كل 30 دقيقة.
- **القرار:** `Access Token` عمره 30 دقيقة يُحمّل في جسم الاستجابة (يُستخدم في رأس `Authorization`)؛ `Refresh Token` عمره 30 يومًا لا يصل جسم الاستجابة إطلاقًا — يُحمّل حصريًا في كوكي `HttpOnly`. عند إعادة استخدام رمز تحديث مُبطل، تُبطل كل الرموز النشطة لذلك المستخدم فورًا ويُدوّر `SecurityStamp` (يُبطل كل `Access Token` الحالية أيضًا، لا فقط رموز التحديث).
- **البدائل المرفوضة:** رمز تحديث في جسم الاستجابة/`localStorage` (رُفض: عرضة لسرقة عبر `XSS`)؛ رمز وصول طويل الأمد بلا تحديث (رُفض: نافذة خطر أطول بلا آلية إبطال سريعة).

- **الأثر:** يتطلب تنسيقًا صارمًا مع الواجهة الأمامية حول كوكيز SameSite/Secure عبر النطاقات المختلفة (انظر ADR-007).

PostgreSQL — ADR-003 مع امتداد PostGIS كقاعدة بيانات تشغيلية وحيدة

- **الحالة:** مقبول ومطبق.
- **السياق:** البحث عن عقارات يتطلب استعلامات جغرافية (القرب من نقطة، ضمن نطاق/حي) بجانب استعلامات علائقية عادية (تصفية بالسعر، النوع، عدد الغرف).
- **القرار:** استخدام PostgreSQL + PostGIS بدل قاعدة علائقية عادية + خدمة بحث جغرافي منفصلة، أو NoSQL + فهرسة جغرافية خارجية.
- **البدايل المرفوضة:** محرك بحث منفصل (Elasticsearch/OpenSearch مع طبقة جغرافية) (رُفض حاليًا: يضيف مكوّن بنية تحتية إضافيًا وتعقيد مزامنة بيانات لحجم البيانات الحالي)؛ قاعدة NoSQL بحقول جغرافية (رُفض: يضخّ باتساق ACID المطلوب لبيانات مالية/قانونية مثل الأسعار والتراخيص).
- **الأثر:** كل استضافة يجب أن تدعم امتداد PostGIS (وليس أي PostgreSQL عام) — قيد فعلي على اختيار مزوّد الاستضافة (Render يدعمه فعليًا عبر صورة postgres/postgis).

Cloudinary — ADR-004 كمزوّد وحيد لتخزين ومعالجة الوسائط

- **الحالة:** مقبول ومطبق، مع بديل اختياري فقط لا إنتاجي.
- **السياق:** الحاجة لرفع/تحويل/حذف صور (عقارات، صور رمزية، شعارات) بأحجام وأنواع متعددة دون بناء خط أنابيب معالجة صور داخلي.
- **القرار:** واجهة IMediaStorageService واحدة في Application، بتنفيذ إنتاجي وحيد (CloudinaryMediaStorageService) وتنفيذ اختياري منفصل (StagingSmokeMediaStorageService)، للحفاظ على استقلالية اختبارات الدخان عن حصة Cloudinary الحقيقية).
- **البدايل المرفوضة:** تخزين محلي على القرص/S3 مباشرة (رُفض: يتطلب بناء تحويل/تحسين الصور يدويًا).
- **الأثر المعروف:** خطة Cloudinary المجانية تدعم "بيئة منتج" واحدة فقط — عزل Staging عن Production هو عزل بيانات اعتماد (API Key) لا عزل تخزين فعلي؛ صور كلتا البيئتين تهبط في نفس المجلدات المبرمجة صراحة في الكود.

ADR-005 — عزل أحمال العمل على Redis (مُعاد صياغته من docs/architecture/adr-redis-workload-isolation.md)

- **الحالة:** مقترح؛ التنفيذ (مزوّدون منفصلون) لم يكتمل بعد.
- **السياق:** Redis واحد يخدم اليوم أربعة أحمال غير مترابطة: تحديد المعدّل، التخزين المؤقت للمخرجات، تسريع Security Stamp/التخزين المؤقت الموزّع العام، وSignalR backplane الاختياري. انقطاع أو تشبّع Redis واحد يمكن أن يمسّ الأمان (تحديد المعدّل) والتوفر معًا في آن واحد.
- **القرار:** الهدف طويل الأمد هو مزوّدون منفصلون (أو عزل مكافئ) لكل حمل، خصوصًا فصل تحديد المعدّل الحساس أمنيًا عن بقية الأحمال. أرقام قواعد بيانات Redis المنطقية (SELECT 0..15) لا تُعتبر عزلاً كافياً لأنها لا تعزل المعالج أو الذاكرة أو الاتصالات أو التوافر.

- **البدايل المرفوضة: Redis** واحد مشترك لكل الاحمال بشكل دائم في بيئة إنتاج ناضجة (مقبول مؤقتًا اليوم، مرفوض كهدف نهائي)؛ الاعتماد على أرقام قواعد بيانات منطقية فقط كحدود عزل.
- **خطة الترحيل:** الإبقاء على الإعداد المشترك في غير الإنتاج → التحقق الإلزامي من أدلة توفر عالٍ (HA) في الإنتاج → إدخال متغيرات اتصال منفصلة لكل حمل → نقل تحديد المعدل أولاً إلى Redis عالي الأولوية منفصل.

ADR-006 — المكتب العقاري (Agency) تجميع تنظيمي وليس عزل مستأجرين (Multi-Tenancy)

- **الحالة:** مقبول ومطبّق، بقيد معروف موثّق صراحة.
- **السياق:** الحاجة لتجميع مستخدمين تحت "مكتب عقاري" (اسم، شعار، صفحة عامة، عضوية) دون إعادة تصميم نموذج ملكية العقار الحالي (Property.OwnerId) القائم منذ بداية المشروع.
- **القرار:** Agency كيان تجميعي إضافي فقط: Property.OwnerId يبقى كما هو؛ UserAccount.AgencyId اختياري يسجّل الانتماء التنظيمي فقط. لا يوجد **Global Query Filter** على AgencyId ولا عزل صفوف صارم — كل استعلام موجود يستمر بالعمل دون تعديل، وأي فصل بين بيانات مكتبين يتم عبر مُسندات (Predicates) صريحة في نقاط الاستخدام الفعلية فقط.
- **البدايل المرفوضة:** عزل مستأجرين كامل من اليوم الأول (Global Query Filter + IAgencyContext) سياقي) — رُفض بوعي لأنه تغيير أكبر بكثير من نطاق الحاجة الحالية، ويتطلّب "Design Spike" منفصلاً وفق مراجعة المعمارية الداخلية.
- **الأثر المعروف والمقبول:** معالج ينسى إضافة مُسند AgencyId صراحة يمكن أن يسرّب بيانات عبر حدود مكتبين — هذا خطر حقيقي مقبول اليوم، مع مسار ترقية معروف وواضح إن استُدعيت الحاجة لعزل صارم.

ADR-007 — توكن CSRF مُعاد في جسم JSON، لا الاعتماد على قراءة كوكي عبر النطاقات فقط

- **الحالة:** مقبول ومطبّق عبر مستودعي الوجهتين معًا (تنسيق متعدد المستودعات).
- **السياق:** الواجهة الأمامية (realestateworld.world على Netlify) والخلفية (نطاق onrender.com) لا يشتركان بأي لاحقة نطاق مسجّلة (**Registrable Domain**) في staging/production آلية Angular المدمجة (withXsrfConfiguration) تعتمد على قراءة كوكي غير HttpOnly من document.cookie على أصل (Origin) الواجهة الأمامية — وهذا **مستحيل تقنيًا** عبر نطاقات لا تشترك أي جزء، بصرف النظر عن أي إعداد Domain على الكوكي. حدث هذا فعليًا كعطل إنتاجي حقيقي: تسجيل خروج قسري لكل مستخدم كل 30~ دقيقة (فشل CSRF على تحديث الرمز) لأن الواجهة لم تكن ترسل رأس X-XSRF-TOKEN أصلاً.
- **القرار:** GET /api/security/csrf-token يُعيد التوكن في جسم استجابة { csrfToken } **JSON** بالإضافة إلى كوكي XSRF-TOKEN قابل للقراءة من جافاسكربت (يبقى يعمل محليًا حيث الأصلان يشتركان في localhost). الواجهة الأمامية تخزّن القيمة من الجسم في خدمة داخلية (CsrfTokenService) وترسلها كرأس X-XSRF-TOKEN عبر Interceptor موحد، بما يشمل نقطتي الاستدعاء اللتين تتجاوزان سلسلة الـ Interceptors عمدًا.
- **البدايل المرفوضة:** الاعتماد فقط على آلية Angular المدمجة (withXsrfConfiguration) — **مُختبرة فعليًا وتبيّن أنها تعمل محليًا فقط** وتفشل صامتة في staging/production؛ إسقاط CSRF كليًا (رُفض: يفتح الباب لهجمات تزوير طلب عبر الموقع على نقاط نهاية تعتمد على الكوكي).
- **الأثر:** أي تغيير مستقبلي في نطاقي الاستضافة (توحيدهما تحت نطاق مشترك، مثلاً) يفتح الباب لإعادة تقييم هذا القرار، لكن لا حاجة ملحة له اليوم.

Render.com — ADR-008 كمنصة استضافة، بيئتي Staging و Production معزولتين منطقيًا داخل مشروع واحد

- **الحالة:** مقبول ومطبّق.
- **السياق:** الحاجة لبيئة اختبار قبل-الإنتاج (Staging) تحاكي الإنتاج بأمانة كافية (نفس صورة الحاوية، نفس مخطط قاعدة البيانات) دون تكلفة تشغيل بنية تحتية منفصلة بالكامل.
- **القرار:** مشروع Render واحد (prj-d5b6uk21i9vc73bhpe20) يحتوي بيئتين منفصلتين منطقيًا: قاعدتا بيانات مستقلتان تمامًا، خدمتا ويب منفصلتان، و Redis منفصل لكل بيئة (Upstash للإنتاج، Valkey على Render لـ Staging). النشر إلى Staging عبر Deploy Hook يدوي بعد نجاح بوابة production-gate.yml فقط — لا نشر تلقائي عند كل Push.
- **البدايل المرفوضة:** حساب/مشروع Render منفصل بالكامل لكل بيئة (رُفض: تكلفة إدارية أعلى بلا فائدة عزل إضافية حقيقية بما أن Render نفسه يعزل موارد كل بيئة على مستوى الخدمة أصلًا)؛ نشر تلقائي مباشر من فرع master إلى Staging عند كل Commit (رُفض حاليًا: يفضل بوابة تحقق ناجحة أولًا).
- **الأثر المعروف:** خطة "Free" Render "لموارد Staging تفرض قيودًا حقيقية (بلا نسخ احتياطي لقاعدة البيانات، صلاحية ~30 يومًا، بلا Shell access) موثقة في القسم 2.3 وتنعكس مباشرة على القدرة على تشخيص مشاكل عنوان IP خلف الوكيل (انظر ADR الأمني في القسم 4.4).

6. اعتبارات الجودة (Quality Attribute Considerations)

6.1 الأداء

الهدف (SLO)	المؤشر (SLI)	نافذة القياس
التوفر (Availability)	نسبة الطلبات غير 5xx إلى إجمالي الطلبات $\leq 99.5\%$	متحركة، 30 يومًا
الاستجابة (Latency)	نسبة الطلبات المكتملة ضمن $500ms \leq 95\%$	متحركة، 30 يومًا
المصادقة (Authentication)	نسبة عمليات المصادقة الناجحة إلى محاولات صالحة $\leq 99.0\%$	متحركة، 30 يومًا

قواعد تسجيل Prometheus (Recording Rules) تُطبّق هذه المؤشرات، مع تنبيهات **Fast-Burn** ($\times 14.4$) و **Slow-Burn** ($\times 2$) لميزانية خطأ التوفر — وليس متوسطات بسيطة، بل **مئينات (p95/p99) من هستوغرامات فعلية**، وهو فارق منهجي متعمّد (المتوسطات تخفي أذيال التوزيع البطيئة).

استراتيجيات التحقق الفعلية في الكود:

- تخزين مؤقت للمخرجات (Output Cache) عبر Redis لنقاط قراءة عامة عالية التكرار.
- تسريع فحص Security Stamp عبر تخزين مؤقت مخصّص (CachedSecurityStampValidator) بدل استعلام قاعدة بيانات على كل طلب مُصادّق.
- فصل عمليات الكتابة الثانوية (إشعارات) عن المسار الحرج للاستجابة (Best-Effort، انظر القسم 4.3).

6.2 قابلية التوسع

- **بلا حالة على مستوى العملية (Stateless Compute):** أي نسخة من PropertyApi قابلة للاستبدال فورًا دون فقدان جلسات أو حالة — الحالة الوحيدة الحية في الذاكرة هي ذاكرة تخزين مؤقت قصيرة العمر قابلة لإعادة البناء.
- **SignalR Backplane عبر Redis:** يسمح بتشغيل أكثر من نسخة خلفية خلف موازن تحميل مع بقاء الإشعارات الفورية متسقة عبر كل النسخ (SignalR_Provider قابل للتفعيل صراحة عند الحاجة).
- **تحديد المعدّل الموزّع عبر Redis:** يمنع تجاوز حصص الحماية عند تشغيل أكثر من نسخة (لو كان تحديد المعدّل محليًا في ذاكرة كل نسخة، لكانت كل نسخة تمنح حصّة كاملة منفصلة — ثغرة توسّع أفقي شائعة. يتجنبها هذا التصميم).
- **قيد معروف على التوسّع الأفقي لبيئة Staging تحديدًا:** تعطيل ForwardedHeaders هناك (انظر القسم 4.4) يعني أن تحديد المعدّل حسب IP لا يفرّق فعليًا بين زوّار متعددين خلف نفس حافة Render — قيد بيئة اختبار محدّد، لا قيد معماري عام.

6.3 الأمان

جدول سياسات تحديد المعدّل الفعلية (RateLimitingRegistration.cs) — دليل واضح على نهج "حماية متميزة بحسب حساسية العملية"، لا سياسة واحدة للجميع:

السياسة	الحد	النافذة	العملية المحمية
send-otp	3	15 دقيقة	إرسال رمز OTP للهاتف
verify-otp	5	15 دقيقة	التحقق من رمز OTP
auth-password-reset	3	60 دقيقة	طلب/تنفيذ استعادة كلمة المرور
auth-login	10	1 دقيقة	تسجيل الدخول بالبريد/كلمة المرور
auth-register	5	10 دقائق	إنشاء حساب جديد
auth-refresh	20	5 دقائق	تحديث رمز الوصول
auth-logout	20	5 دقائق	تسجيل الخروج
contact	5	60 دقيقة	نموذج التواصل
visits	10	60 دقيقة	طلب زيارة عقار
reviews	5	24 ساعة	إضافة تقييم
agencies-public	120	1 دقيقة	صفحة المكتب العقاري العامة
public-read	120	1 دقيقة	قراءات عامة أخرى (صور العقار، التقييمات)
public-search / geo-search	60 / 120	1 دقيقة	البحث العام / البحث الجغرافي
shortstay-search	120	1 دقيقة	بحث الإقامة القصيرة
shortstay-booking	10	60 دقيقة	حجز إقامة قصيرة
service-requests	10	60 دقيقة	طلب خدمة
service-request-documents	20	60 دقيقة	رفع مستندات طلب خدمة

راجع القسم 4.5 لتفاصيل المصادقة، التفويض، CSRF، ورؤوس الأمان.

6.4 قابلية الصيانة

- اختبارات معمارية آلية (PropertyApi.Architecture.Tests) تحوّل قواعد التصميم (اتجاه الاعتماد، قائمة نقاط النهاية العامة الموافق عليها) من اتفاقيات موثقة فقط إلى بوابات CI فعلية تُفشل البناء.
- نمط **CQRS** موحد بلا استثناء تقريبًا يعني أن أي مطوّر جديد يتعلّم نمطًا واحدًا (Controller → Command/Query → Handler) وينطبق على كل الوحدات الثلاثين.
- تعليقات الكود توثّق "لماذا" لا "ماذا" فقط — ملاحظة جديرة بالذكر: نسبة كبيرة من التعليقات في هذا الكود تشرح قرارًا أمنيًا أو إصلاح عطل حقيقي (B RELEASE-BLOCKERS-AR.md) بدل وصف الكود حرفيًا، ما يقلّل من "تآكل التوثيق" بمرور الوقت لأن التعليق يبقى صحيحًا حتى لو تغيّر التفصيل التقني حوله.

6.5 قابلية النشر والتعافي من الكوارث

الهدف	القيمة المعتمدة	آلية القياس
RPO (أقصى فقدان بيانات)	≥ 60 دقيقة	نسخ احتياطي بالساعة (الدقيقة 17 من كل ساعة)؛ نسخة غير مرفوعة/مُتحقّق منها/بلا Manifest صالح لا تُحتسب كنقطة تعافٍ صالحة
RTO (أقصى زمن تعافٍ)	≥ 120 دقيقة	من إعلان الحادثة حتى خدمة تشغيلية مُتحقّق منها وإعادة توجيه مروري متحكّم بها؛ مُقاسة فعليًا بسير عمل تعافٍ آلي يُخرج restore-drill-evidence.json
بوابة إصدار للإنتاج	إلزامية	production-gate.yml يجب أن ينجح (بما فيه اختبارات Redis Sentinel HA) قبل اعتبار أي نشر مؤهلًا
صورة حاوية مخفّضة السطح	chiseled	بلا shell/مدير حزم في صورة التشغيل، مستخدم غير جذري

استراتيجية النشر الفعلية: بناء صورة واحدة عبر Dockerfile متعدد المراحل → اجتياز بوابة production-gate (اختبارات + طبولوجيا Redis HA) → نشر يدوي عبر Deploy Hook إلى Staging (خارج نطاق الأتمتة الموثقة هنا) ترقية إلى Production. ترحيلات قاعدة البيانات ليست جزءًا من إقلاع التطبيق في أي بيئة — خطوة منفصلة صريحة عبر tools/PropertyApi.Migrator، ما يمنع سباقًا (Race) بين عدة نسخ تطبيق تحاول ترحيل نفس قاعدة البيانات في آن واحد عند التوسّع الأفقي.

7. المخاطر المعمارية (Architectural Risks)

#	الخطر	الاحتمال	الأثر	التخفيف الحالي / الموصى به
R1	Redis واحد يخدم 4 أحمال غير مترابطة (تحديد معدّل، تخزين مؤقت، Security Stamp, SignalR) — تشبّع أو انقطاع واحد يمسّ الأمان والتوفر معًا	متوسط	عالٍ	ADR-005: خطة ترحيل نحو عزل الأحمال موثقة؛ لم تُنفذ بالكامل بعد

#	الخطر	الاحتمال	الأثر	التخفيف الحالي / الموصى به
R2	لا عزل صفوف صارم بين المكاتب العقارية (Agencies) — معالج جديد قد ينسى مُسنَدًا صريحًا فيسرّب بيانات عبر حدود مكتبين	منخفض حاليًا (حجم الميزة محدود)، يرتفع مع نمو الميزة	عالي إن تحقق	ADR-006: قيد موثّق بوعي؛ مسار ترقية معروف (Global Query Filter + IAgencyContext) غير منقّد
R3	تحديد المعدّل حسب IP على Staging غير فعال فعليًا (IP الحافة الموحد ل Render)	مؤكّد الحدوث (خطر معروف)	منخفض (Staging) فقط، ليس (Production)	مقبول صراحة كقرار مؤرّخ؛ يُعاد النظر عند ترقية خطة Render أو الحاجة لاختبار متزامن حقيقي
R4	عزل Cloudinary/Staging- Production بيانات اعتماد فقط لا تخزين — صور الاختبار وصور الإنتاج تتشارك نفس المجلدات المنطقية	منخفض	متوسط (احتمال تلوث بيانات وسائط بين بيئتين، لا تسريب أمني مباشر)	لا تخفيف مطبّق حاليًا؛ خيار مستقبلي: مجلد مبنّي على اسم البيئة بدل ثابت
R5	مستودعا الواجهتين منفصلان تمامًا بلا فحص نوع (Type-Checking) مشترك لعقد HTTP	متوسط (يتطلب تنسيقًا يدويًا مستمرًا)	متوسط-عالي (عطّل CSRF التاريخي مثال فعلي وقع)	لا أداة آلية: الاعتماد على اختبارات دخان Staging (PropertyApi.StagingSmokeTests) للكشف بعد النشر
R6	خطة "Free Render" ل Staging: بلا نسخ احتياطي، صلاحية قاعدة بيانات ~30 يومًا، بلا Shell access للتشخيص	مؤكّد (قيد بنية تحتية حالي)	منخفض-متوسط (فقط Staging)	مقبول كتوازن تكلفة/فائدة لبيئة اختبار؛ لا ينطبق على Production
R7	ترجيلات قاعدة البيانات يدوية بالكامل عبر أداة منفصلة، لا جزء من الإقلاع الآلي	منخفض (بتصميم متعمّد)	متوسط إن نُسيت الخطوة قبل نشر يعتمد على مخطط جديد	إجراء تشغيلي موثّق (Runbook)؛ يعتمد على انضباط عملية النشر اليدوية
R8	لا مسار Webhook وارد موثّق من أي مزود خارجي (Cloudinary/Resend/SMS) — أي تغيير حالة خارجي (فشل بريد لاحق، إلخ) لا يُعاد للنظام تلقائيًا	غير مؤكّد (خارج نطاق هذا التحليل)	منخفض-متوسط	يستحق تحققًا منفصلًا خارج نطاق هذه الوثيقة

8. الخاتمة

8.1 الملخص

PropertyApi نظام واجهة خلفية ناضج نسبيًا، مبني بانضباط معماري ملحوظ: فصل طبقات مفروض آليًا (لا اتفاقًا فقط)، نمط CQRS موحد بلا استثناء تقريبًا عبر ثلاثين وحدة تحكم، ومركزية متكررة لمنطق التفويض والتحقق الحساس. الأمان ليس طبقة مضافة لاحقًا بل قرارات متكررة ومقصودة في كل معالج (عدم تسريب معلومات، عدم

الثقة بمدخلات العميل الحساسة، فحص فعلي لمحتوى الملفات لا امتدادها فقط). أبرز نقاط القوة: بوابات CI آلية حقيقية (معمارية، Redis HA، اختبارات دخان)، مراقبة مدمجة منذ الإقلاع، وتوثيق داخلي غني بـ "لماذا" وراء كل قرار. أبرز القيود المعروفة والمقبولة بوعي (لا عيوب مخفية): عدم اكتمال عزل أحمال Redis (ADR-005)، عدم وجود عزل صفوف صارم بين المكاتب العقارية (ADR-006)، واعتماد كامل على تنسيق يدوي بين مستودعي الواجهتين المنفصلين لأي تغيير في عقد HTTP.

8.2 المراجعات المستقبلية

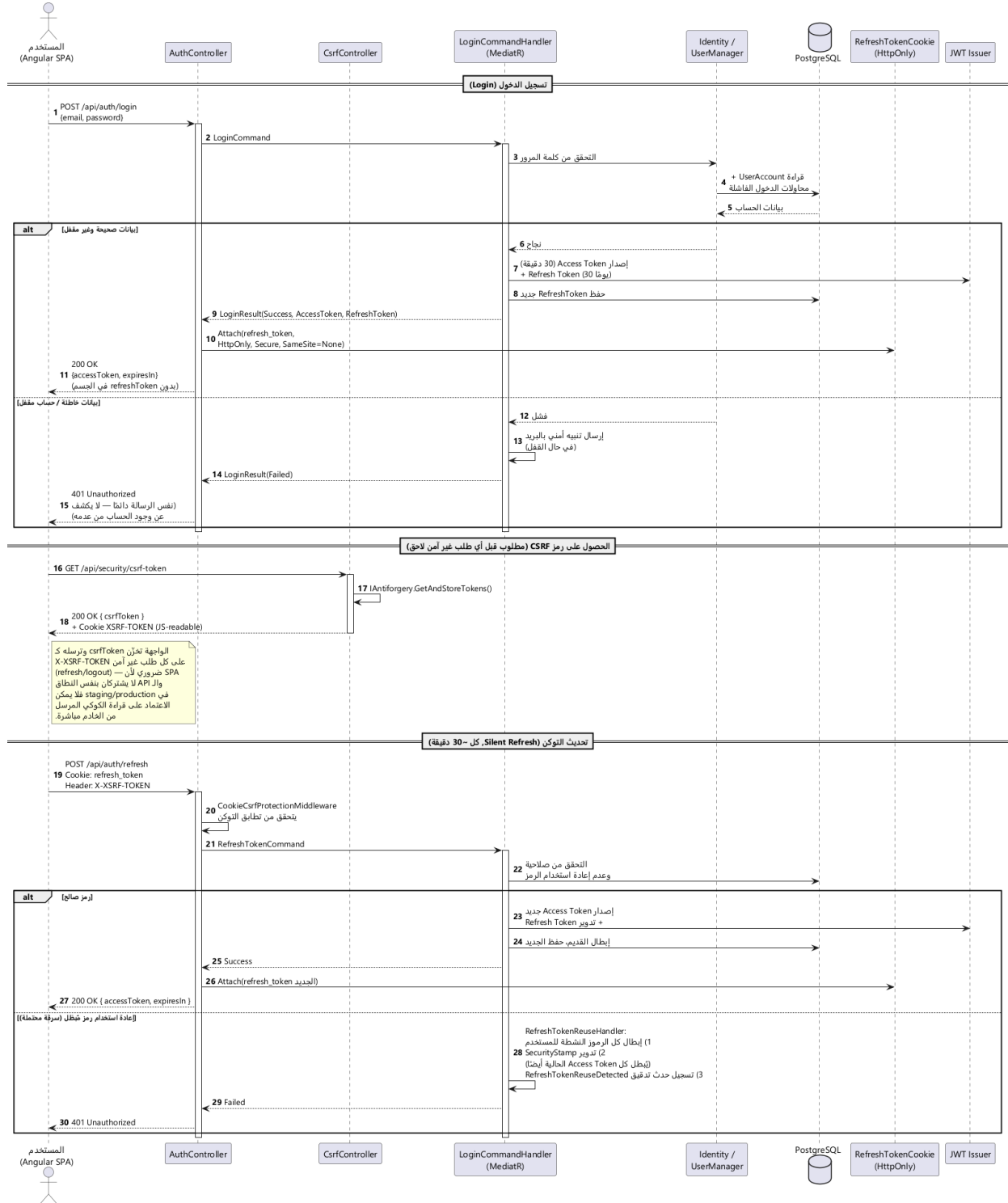
يجب مراجعة هذه الوثيقة عند وقوع أي من التالي:

- تنفيذ فعلي لخطة عزل أحمال Redis (ADR-005) أو تغيير مزود Redis.
 - أي قرار بترقية عزل المكاتب العقارية إلى عزل صفوف صارم (ADR-006).
 - تغيير منصّة الاستضافة (خروج من Render) أو توحيد نطاقات الواجهتين الأمامية والخلفية.
 - إضافة وحدة عمل جديدة كبيرة (سوق جديد، نوع مستخدم جديد) تُدخل نمط تفويض مختلفًا عن النمط الموحد الحالي.
 - أي تغيير في أهداف SLO/RPO/RTO المعتمدة حاليًا.
- يُوصى بمراجعة دورية لا تقل عن مرة كل ربع سنة، بالتوازي مع أي تدقيق أمني أو تدريب تعافٍ من كوارث (Recovery Drill).

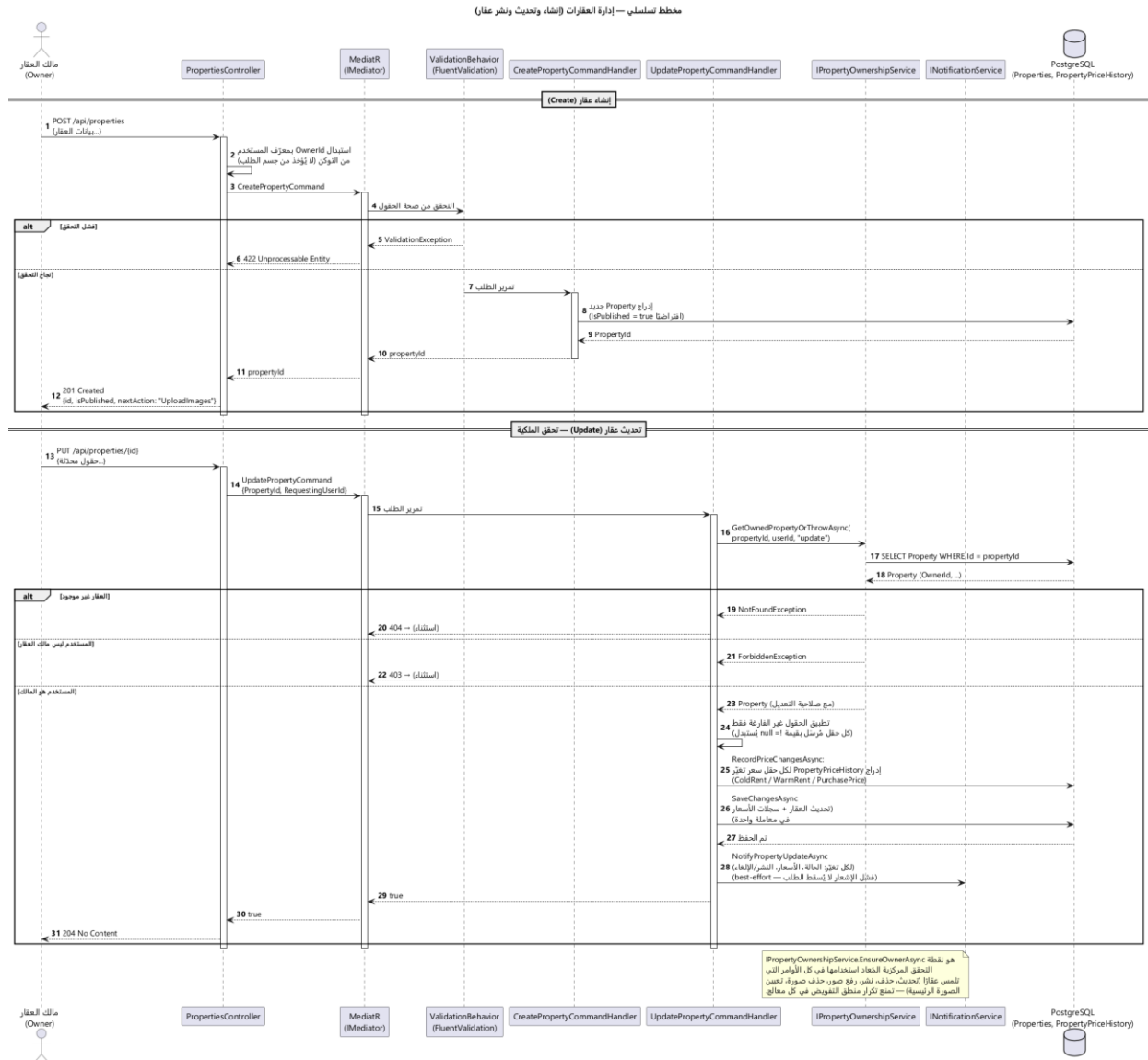
9. الملحق — المخططات التسلسلية للعمليات الجوهرية

9.1 المصادقة (Authentication)

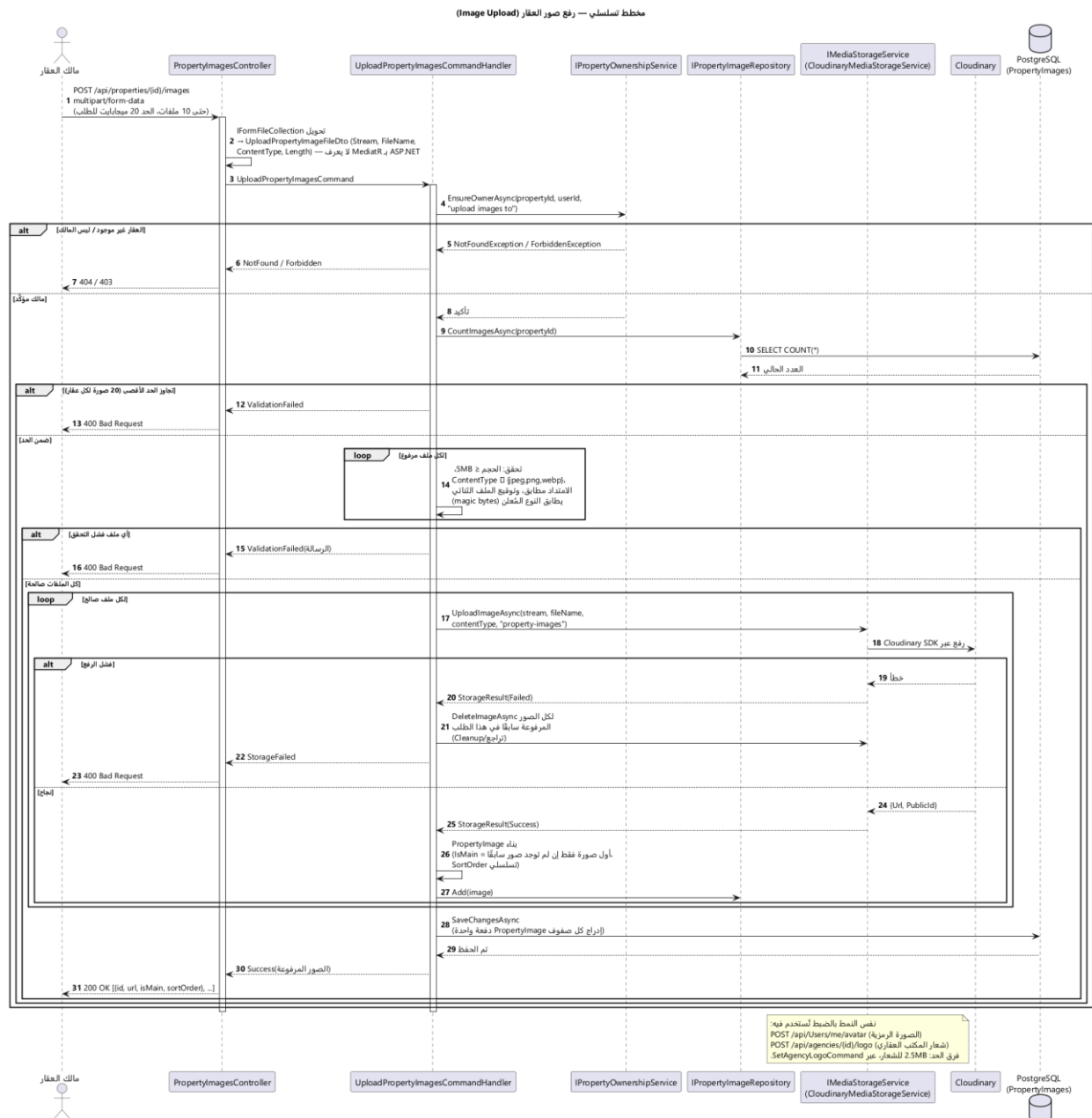
مخطط تسلسلي — المصادقة (Login + Refresh + CSRF)



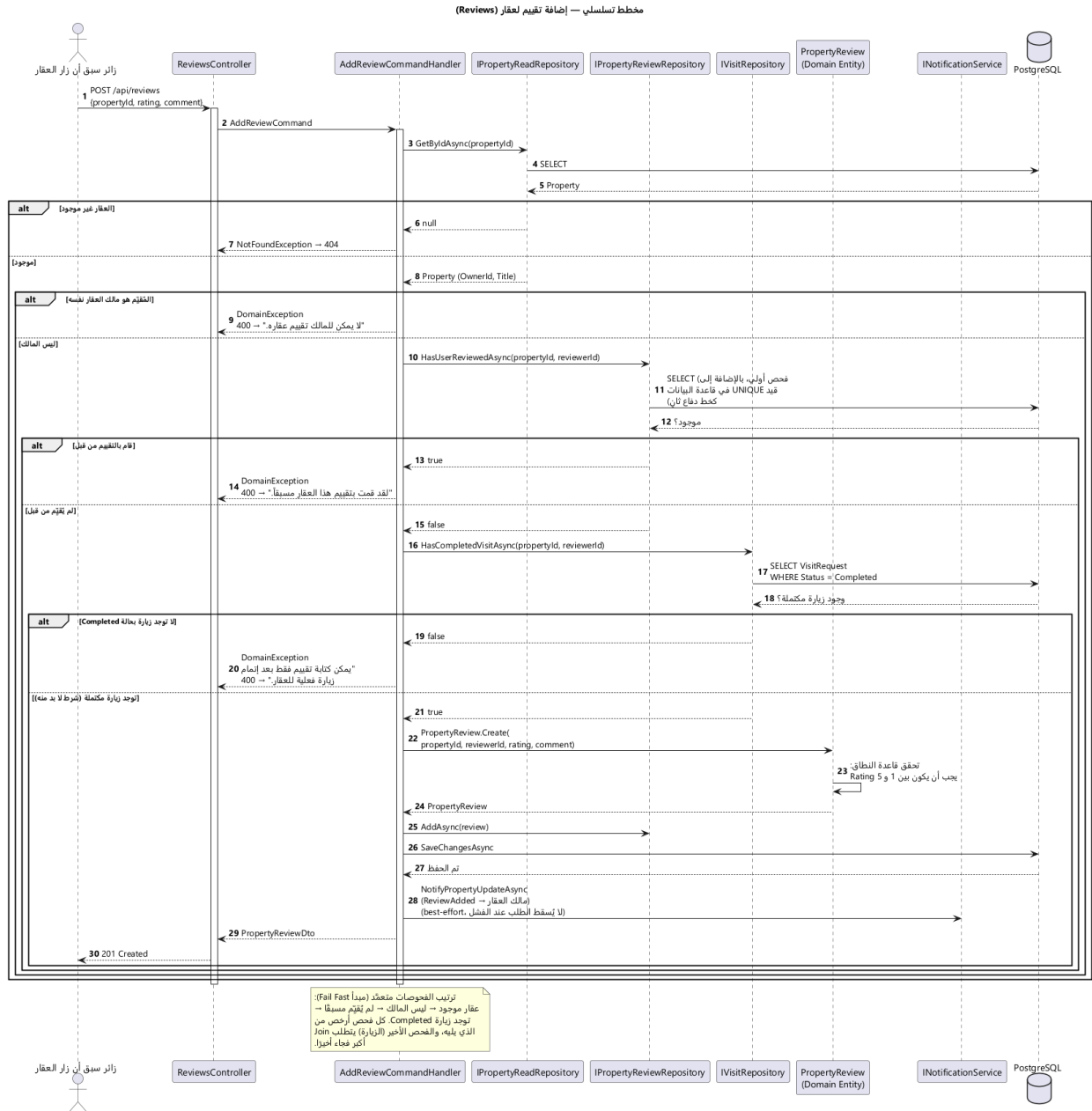
9.2 إدارة العقارات (Property Management)



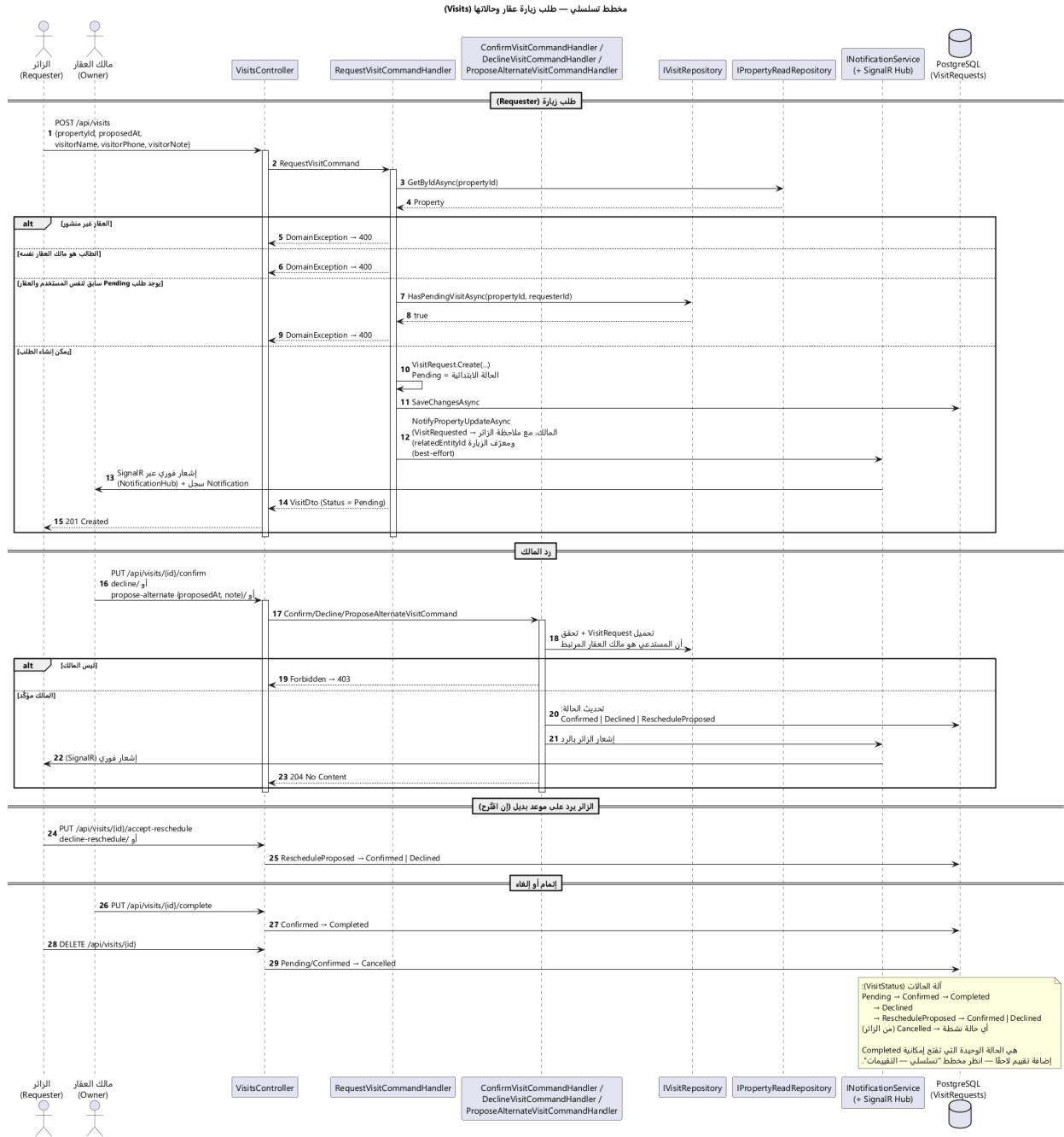
9.3 رفع الصور (Image Upload)



9.4 التقييمات (Reviews)



9.5 الزيارات (Visits)



*نهاية الوثيقة. جميع المخططات متوفرة بصيغة PlantUML قابلة للتعديل في

.docs/architecture/ARD/diagrams/.puml